

4. To Split Input into Tokens, Identify Delimiters, Handle Whitespace, Create Token Structures, Validate Token Streams, Debug Parsing Output.

To Design Parser Logic, Generate Parse Trees, Validate Syntax, Detect Errors, Handle Empty Commands, Produce Execution Structures.

```
GNU nano 8.7.1
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <ctype.h>

#define MAX_INPUT 1024
#define MAX_TOKENS 100

int main()
{
    char input[MAX_INPUT];
    char tokens[MAX_TOKENS][MAX_INPUT];
    int tokenCount = 0;

    printf("==== Mini Shell Quote Parser =====\n");
    printf("Type 'exit' to quit.\n\n");

    while (1)
    {
        printf("MiniShell> ");

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
            break;

        if (strlen(input) == 0)
            continue;

        tokenCount = 0;

        int i = 0;

        while (input[i] != '\0')
        {
            while (isspace(input[i]))
                i++;

            if (input[i] == '\0')
                break;
```

```

if (input[i] == '\\0')
    break;

int j = 0;

if (input[i] == '\\')
{
    i++;

    while (input[i] != '\\ ' && input[i] != '\\0')
    {
        tokens[tokenCount][j++] = input[i++];
    }

    if (input[i] == '\\ ')
        i++;
}
else if (input[i] == "\\")
{
    i++;

    while (input[i] != "\\ " && input[i] != '\\0')
    {
        if (input[i] == '$')
        {
            i++;

            char var[100];
            int k = 0;

            while (isalnum(input[i]) || input[i] == '_')
            {
                var[k++] = input[i++];
            }

            var[k] = '\\0';

            char *value = getenv(var);

            if (value != NULL)
            {
                strcpy(&tokens[tokenCount][j], value);
                j += strlen(value);
            }
        }
    }
}

```

```

        if (value != NULL)
        {
            strcpy(&tokens[tokenCount][j], value);
            j += strlen(value);
        }
    }
    else
    {
        tokens[tokenCount][j++] = input[i++];
    }
}

if (input[i] == '\\')
    i++;
else
{
    while (input[i] != '\\0' && !isspace(input[i]))
    {
        tokens[tokenCount][j++] = input[i++];
    }

    tokens[tokenCount][j] = '\\0';
    tokenCount++;
}

printf("\\nParsed Tokens:\\n");

for (int x = 0; x < tokenCount; x++)
{
    printf("Token %d : %s\\n", x + 1, tokens[x]);
}

printf("-----\\n");
}

printf("Shell Closed.\\n");

return 0;
}

```

```

thumu_hemanth_kumar@Hemanths-TUF-Laptop:~/OSSkill$ ls
skill1 skill1.c skill1b skill1b.c skill2 skill2.c skill3 skill3.c
thumu_hemanth_kumar@Hemanths-TUF-Laptop:~/OSSkill$ nano skill4.c
thumu_hemanth_kumar@Hemanths-TUF-Laptop:~/OSSkill$ gcc skill4.c -o skill4
thumu_hemanth_kumar@Hemanths-TUF-Laptop:~/OSSkill$ ./skill4

```

```

===== Mini Shell Quote Parser =====
Type 'exit' to quit.

```

```
MiniShell> ls -l
```

```
Parsed Tokens:
```

```
Token 1 : ls
```

```
Token 2 : -l
```

```
-----
MiniShell> |
```